Sistema Bicing: Modelado Relacional, Integración de Datos y Estimación de Ocupación mediante Difusión en Grafo
para el Proyecto de Teoría para Bases de Datos en la Universitat de Barcelona
por **Javier Rivera Romeu, Albert Villanueva Kosoy, Arnau Pascual Aranda y Yasmina Dermouh Dermouh.**
Semestre de Primavera, 2026.

ÍNDICE.

1. Introducción 2
2. Objectivos 3
3. Características de los Datos 3
4. Modelo ER 4
5. Transformación del Modelo ER en Tablas del Modelo Relacional 8
6. Álgebra Relacional 9
7. Predicción de ocupación mediante difusión en grafo 9
8. Normalización del Modelo Relacional 10
9. Implementación 11
10. Análisis de Datos 12
11. Conclusiones 18
12. Contribución de los miembros del grupo y uso de la IA 19

El código se encuentra en https://github.com/jrriverarомeu/ProyectoBicing

1. Introducción

Bicing es el servicio público de bicicleta compartida del Ayuntamiento de Barcelona, en funcionamiento desde 2007 y gestionado por Barcelona de Serveis Municipals (B:SM). Con una red que integra más de 500 estaciones y un parque móvil de aproximadamente 7000 bicicletas, es uno de los sistemas de referencia en Europa. Barcelona ocupa el primer puesto continental en densidad de estaciones por kilómetro cuadrado y en nivel de rotación de bicicletas. El servicio supera los 18 millones de desplazamientos anuales y moviliza a más de 60.000 personas cada día laborable, lo que genera un volumen continuo de datos heterogéneos (de usuarios, bicicletas y estaciones) cuya gestión analítica resulta esencial para garantizar la disponibilidad del servicio y optimizar su rendimiento.

La idea central del proyecto es transformar esos datos heterogéneos en una base de datos relacional que permita estudiar el comportamiento del sistema y estimar la ocupación futura de las estaciones. El interés reside en construir una estructura que facilite consultar tendencias, comparar estaciones entre sí y explotar la dimensión espacial y temporal del sistema.

El proyecto parte de la siguiente hipótesis: la ocupación de una estación no depende solo de su capacidad física, sino también de su posición dentro de la red ciclista, de su proximidad a otras estaciones, de la conectividad de los carriles bici que la rodean y de factores territoriales como la altitud o el barrio en el que se encuentra. Esa hipótesis es la que justifica tanto el modelado relacional como la construcción de un grafo de estaciones.

1.1 Hipótesis de Trabajo

La idea es utilizar un grafo de flujos para predecir la ocupación de una estación.

La hipótesis principal del proyecto sostiene que la disponibilidad de bicicletas en una estación puede aproximarse de manera más precisa si se considera la interacción de tres tipos de variables.

- La propia evolución histórica del estado de la estación permite identificar si tiende a llenarse o a vaciarse en determinadas franjas horarias.
- La relación espacial con las estaciones vecinas y con la red de carriles bici, que condiciona los flujos de entrada y salida, aunque estos no se observen de manera explícita.
- La información territorial, en particular la altitud. En una ciudad con relieve tan marcado como Barcelona, las estaciones situadas en cotas elevadas tienden a perder bicicletas con más facilidad cuando la dinámica de desplazamiento favorece los trayectos cuesta abajo, mientras que las estaciones en zonas bajas acumulan más retornos de forma sistemática.

La ocupación futura de una estación puede estimarse mejor si se concibe la red como un sistema interdependiente y no como un conjunto de puntos aislados. Por eso el modelo no se limita a almacenar observaciones individuales, sino que incorpora explícitamente una estructura de vecindad entre estaciones construida a partir de distancias geográficas y de la infraestructura ciclista real. Esa estructura de vecindad será la base del componente predictivo desarrollado en la última sección del proyecto.

2. Objetivos

El objetivo general del proyecto es transformar los datos heterogéneos del sistema Bicing en una base de datos relacional que permita analizar el comportamiento de las estaciones y estimar su ocupación futura. Para alcanzarlo, el trabajo se ha ido dividiendo en objetivos concretos y modulares.

1. El **modelado de datos**: definir un modelo conceptual y relacional que represente correctamente la información de estaciones, estados temporales y red de carriles bici, asegurando la integridad referencial y la normalización del esquema.
2. La **integración de fuentes heterogéneas**: recoger y transformar datos en diferentes formatos (JSON y GeoJSON) en tablas relacionales normalizadas, resolviendo los problemas de anidamiento, codificación geoespacial y coherencia.
3. La **creación de un grafo de estaciones**: representar las conexiones espaciales y de flujo entre estaciones a partir de la infraestructura ciclista real, de modo que pueda aplicarse sobre él un modelo de predicción basado en redes.
4. La **predicción de ocupación**: implementar un modelo de difusión sobre el grafo que estime el estado futuro de cada estación considerando factores temporales, espaciales y territoriales. El quinto es **facilitar consultas y análisis**: permitir la realización de consultas mediante álgebra relacional para explorar tendencias, comparar estaciones y generar estadísticas agregadas sobre el uso de la red.

3. Características de los datos

Los datos provienen de tres fuentes abiertas:

1. **Información de estaciones (station_information)**: JSON con atributos estáticos de cada estación: ID, nombre, coordenadas geográficas, altitud, capacidad y distrito/barrio.
2. **Estado de estaciones (station_status)**: JSON con snapshots temporales del número de bicicletas disponibles, anclajes libres y estado operativo. Incluye desglose de bicicletas mecánicas y eléctricas.
3. **Red de carriles bici (bike_lanes)**: GeoJSON con geometrías lineales (LINESTRING) que representan la infraestructura ciclable.

Cada fuente se procesa para:
- Normalizar estructuras anidadas en tablas relacionales.
- Transformar coordenadas geográficas y geometrías en tipos espaciales de MySQL.
- Asociar estaciones a carriles bici para construir el grafo de adyacencia.

3.1. Recogida de Datos

La recogida de datos se apoya en tres fuentes abiertas. La primera es el conjunto de datos de información de estaciones del Bicing* que proporciona atributos estáticos como identificador, nombre, coordenadas, altitud y capacidad. La segunda es el conjunto de estado de estaciones* que recoge snapshots temporales con el número de bicicletas disponibles, el número de anclajes libres y el estado operativo de la estación. La tercera es la red de carriles bici publicada como GeoJSON y adecuada para representar geometrías lineales como segmentos de infraestructura ciclable.

* Hace falta un Token de acceso, mediante registro.

4. Transformación de los Datos y Modelo ER

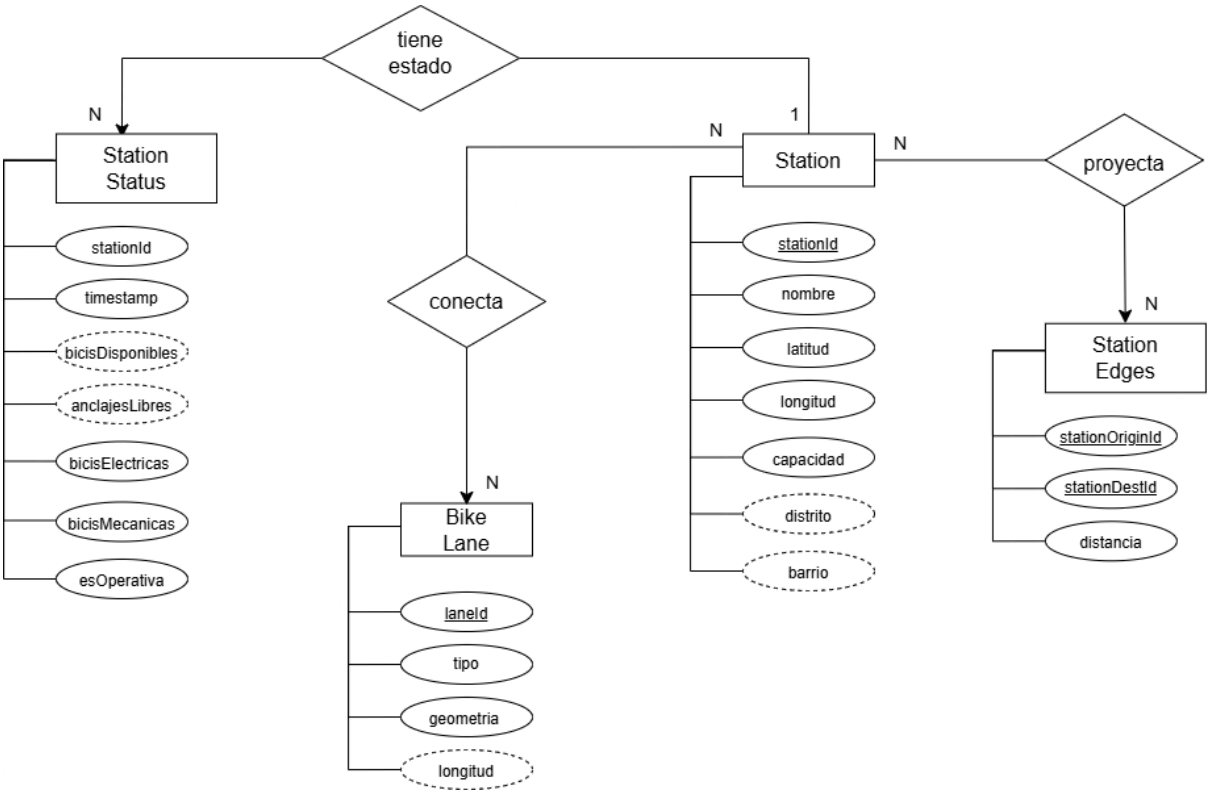
Antes de cargar nada en la base de datos, es necesario definir un modelo conceptual que refleje con claridad qué información se quiere conservar y cómo se relacionan las distintas entidades.

En este proyecto, la entidad principal es la **estación**. Cada estación tiene un **identificador único**, unas **coordenadas geográficas** (latitud y longitud), una **capacidad fija** que representa la máxima capacidad de bicicletas que pueden almacenar y unos atributos territoriales derivados del campo de **localización** (distrito y barrio).

A esta entidad se asocia la entidad de **estado de estación**, dado que una misma estación produce múltiples observaciones a lo largo del tiempo. El estado de estación queda identificado de forma unívoca por la combinación de **stationId** y **timestamp.** El primero indica a qué estación pertenece la observación, y el segundo el momento exacto en que fue registrada. Indica el número de **bicis_disponibles**, **anclajesLibres**, **bicisElectricas**, **bicisMecanicas** y el estado de la estación **esOperativa**.

La relación entre ambas entidades es de uno a muchos: una estación puede aparecer en muchos registros de estado, pero cada registro pertenece a una sola estación.

El tercer bloque conceptual lo forma la red de **carriles bici.** Como los carriles no describen estaciones, sino segmentos de infraestructura lineal, pueden modelarse como otra entidad independiente, con **su identificador único**, el **tipo** de carril, su **geometría** y los **atributos descriptivos** que aparezcan en el GeoJSON. Su papel principal es ayudar a construir relaciones espaciales entre estaciones, por eso el modelo puede complementarse con una relación derivada de adyacencia, donde cada tupla representa que dos estaciones están conectadas o próximas dentro de un umbral espacial razonable.



4.1 Transformación de JSON a SQL.

Los datos de las tres fuentes no son tabulares. Los feeds del Bicing se distribuyen como JSON con estructuras anidadas y el dataset de carrils bici como GeoJSON con geometrías lineales. El proceso ETL debe aplanar esas estructuras y convertirlas en filas y columnas antes de insertarlas en el sistema relacional.

El pipeline se organiza en cinco scripts Python con responsabilidades separadas:

- config.py centraliza las URLs, el token de autenticación y las credenciales de MySQL.
- fetcher.py implementa la descarga añadiendo la cabecera Authorization a cada petición, siguiendo el mecanismo para el acceso a los datos del portal Open Data Barcelona.
- run_etl.py encadena los tres módulos en el orden correcto respetando las dependencias de clave foránea.
- etl_station_information.py, etl_station_status.py y etl_carril_bici.py leen cada fuente, aplanan las estructuras anidadas y generan las sentencias SQL de inserción.

Destacan dos decisiones de transformación con relevancia analítica. La primera afecta al feed de estado: el JSON original agrupa el desglose por tipo de bicicleta bajo una clave anidada num_bikes_available_types, que contiene a su vez los campos mechanical y ebike. Si ese "subobjeto" se almacenara tal cual en una columna de tipo JSON, cada consulta que necesitara distinguir entre bicicletas mecánicas y eléctricas tendría que parsear ese JSON en tiempo de ejecución mediante funciones como JSON_EXTRACT(). En su lugar, el script extrae ambos valores durante la transformación y los almacena como columnas escalares independientes (mechanical_bikes, electrical_bikes) de modo que el acceso es directo y las consultas analíticas pueden filtrar, agregar o comparar ambos tipos sin coste adicional de parseo.

La segunda decisión afecta a las geometrías. Tanto las coordenadas de las estaciones como los segmentos de carril bici se insertan especificando explícitamente el SRID 4326, que corresponde al sistema de referencia WGS84 (el mismo que usa el GPS y el estándar GeoJSON). Es necesario hacerlo así puesto que MySQL solo puede calcular distancias métricas reales y realizar operaciones espaciales siempre que haya una coherencia entre las dos geometrías, es decir, que estén definidas en el mismo sistema de referencia. Si las geometrías se insertaran sin SRID o con SRIDs distintos, ST_Distance() devolvería valores en grados en lugar de metros, y operaciones como ST_Intersection() producirían resultados incorrectos. Declarar el SRID en la definición de la columna del esquema y en cada inserción garantiza que MySQL aplique automáticamente la métrica correcta en cualquier consulta espacial posterior.

Para ejecutar el pipeline completo (teniendo en cuenta que no hay nada creado anteriormente):

```
> pip3 install requests mysql-connector-python
# Crear el esquema (solo la primera vez)
# Se trata este tema con más detalle en la sección de Implementación.
> mysql -u root -p < schema.sql
# Cargar los datos
#Para que se pueda escribir en la base de datos es necesario actualizar config.py!
> python3 run_etl.py --out-dir ./sql_output
# Construir STATION_EDGES sobre la red ciclista
> mysql -u root -p bicing < station_edges_graph.sql
```

4.2 Creación del grafo

El grafo de estaciones no se construye exclusivamente a partir de la proximidad euclídea entre coordenadas, sino que incorpora información sobre la red de carriles bici y el relieve del terreno como factores que modulan el peso de cada conexión. Los segmentos de carril bici se conciben como indicadores de infraestructura compartida entre estaciones, ya que su presencia en el entorno de dos puntos de servicio sugiere que los desplazamientos entre ambos discurren por

vías ciclistas segregadas. Sin embargo, esos segmentos no conectan directamente las estaciones del Bicing, puesto que el dataset geoespacial describe tramos de vía y no relaciones explícitas entre puntos de servicio. Es necesario, por lo tanto, realizar una transformación adicional que integre ambos elementos en un mismo modelo a través de **reglas de álgebra relacional** aplicadas sobre las tablas del sistema.

La estrategia adoptada construye el grafo en tres fases sucesivas expresadas como consultas SQL. En primer lugar, se genera el conjunto de pares de estaciones candidatas mediante un self-join sobre la tabla STATIONS filtrado por distancia espacial, de modo que solo se consideran pares cuya separación no supere un umbral de vecindad. En segundo lugar, se clasifica cada par en función de si ambas estaciones comparten proximidad a un mismo segmento de BIKE_LANES, lo que aproxima relacionalmente la noción de infraestructura ciclista compartida sin necesidad de routing sobre la red. En tercer lugar, se calcula el peso de cada arista como una expresión compuesta que combina la distancia directa entre estaciones, una penalización aplicada a los pares que no comparten carril bici, y un término adicional proporcional al desnivel positivo entre ambas estaciones. Este último factor recoge la hipótesis central del proyecto sobre el efecto del relieve en los flujos de entrada y salida: las conexiones que implican subida tienen un coste efectivo mayor que las conexiones llanas o descendentes.

En términos de la base de datos, el grafo se almacena como una tabla de adyacencias. Cada fila de STATION_EDGES representa una arista no dirigida con una estación de origen, una estación de destino y un peso numérico que sintetiza los tres factores anteriores. Esta representación es eficiente para consultas relacionales y suficientemente flexible para incorporar pesos adicionales en fases posteriores del proyecto. La decisión de construir el grafo mediante SQL puro, en lugar de delegar ese cómputo en una capa externa de procesamiento geoespacial, responde a un criterio de coherencia con el modelo de datos adoptado: toda la lógica de transformación reside en el mismo sistema relacional que almacena los datos, lo que facilita la trazabilidad y la reproducibilidad del proceso.

Simplificación respecto a la Entrega Parcial

La aproximación más fiel consistiría en proyectar cada estación sobre la red ciclista y calcular el camino más corto entre pares mediante un algoritmo como Dijkstra. Esta solución complicaba muchísimo el procesado de los datos y se alejaba del objetivo del proyecto, por lo tanto, hemos decidido sustituirlo por un conjunto de reglas relacionales que se aproximan a la solución.

En esta solución, dos estaciones están fuertemente conectadas si son geográficamente próximas y si ambas tienen acceso a infraestructura ciclista en su entorno inmediato. La distancia se pondera además por el desnivel, de modo que el peso incorpora el efecto del relieve sobre el coste efectivo del desplazamiento.

La construcción del grafo se articula en tres operaciones encadenadas:

Op1. Pares candidatos por vecindad espacial

Sea S la relación STATIONS. El conjunto de pares candidatos se obtiene mediante un producto cartesiano restringido sobre dos instancias renombradas de S, filtrando por distancia:

$$station_pairs = \pi_{atributos}(\sigma_{a.id < b.id \wedge d(a,b) \leq 500}(\rho_a(S) \times \rho_b(S)))$$

donde la distancia en metros se calcula como:

$$d(a,b) = ST_Distance(a.location,\ b.location) \times 111320 \times cos(\varphi)$$

con φ = 41,4° (puesto que es la latitud de referencia de Barcelona) y los atributos proyectados son station_from, station_to, dist_direct_m y altitude_diff_m = b.altitude − a.altitude.

```sql
USE bicing;
TRUNCATE TABLE STATION_EDGES;

INSERT INTO STATION_EDGES (station_from, station_to, distance_m)
WITH

station_pairs AS (
    SELECT
        a.station_id AS station_from,
        b.station_id AS station_to,
        ROUND(ST_Distance(a.location, b.location) * 111320 * COS(RADIANS(41.4)), 2) AS dist_direct_m,
        -- calculamos diferencia de altitud con COALESCE (para manejar NULL)
        COALESCE(b.altitude, 0) - COALESCE(a.altitude, 0) AS altitude_diff_m
    FROM  STATIONS a
    JOIN  STATIONS b ON a.station_id < b.station_id
    WHERE ST_Distance(a.location, b.location)
          * 111320 * COS(RADIANS(41.4)) <= 500
),
```

Op2. Clasificación por afinidad ciclista

Un par queda marcado como con afinidad ciclista si existe al menos un segmento de BIKE_LANES cuya geometría se sitúa a menos de 150m de ambas estaciones simultáneamente. Relacionalmente es un **semijoin** entre station_pairs y BIKE_LANES:

$$lane_affinity = \pi_{station_from,\, station_to}(station_pairs \ltimes_{\theta} BIKE_LANES)$$

con la condición:

$$\theta: \quad d(S_{from}.location,\ BL.geometry) \leq 150 \wedge d(S_{to}.location,\ BL.geometry) \leq 150$$

Solamente nos interesa clasificar cada par de estaciones y saber si hay un carril bici lo suficientemente cercano a ambas. Como no nos interesa ningún otro atributo de BIKE_LANES, utilizamos semijoin, puesto que devuelve las tuplas de station_pairs para las que existe al menos una tupla coincidente en BIKE_LANES bajo la condición θ, pero sin incorporar columnas de BIKE_LANES al resultado ni multiplicar filas cuando varios segmentos cumplen la condición simultáneamente. Si utilizáramos un join, cada par de estaciones aparecería tantas veces como segmentos de carril satisfacieran la condición. De esta forma, evitamos duplicados.

```sql
-- Operación 2: semijoin con BIKE_LANES (afinidad ciclista)
lane_affinity AS (
    SELECT DISTINCT sp.station_from, sp.station_to
    FROM station_pairs sp
    JOIN BIKE_LANES bl
    ON ST_Distance(
            (SELECT location FROM STATIONS WHERE station_id = sp.station_from),
            bl.geometry
        ) * 111320 * COS(RADIANS(41.4)) <= 150
    AND ST_Distance(
           (SELECT location FROM STATIONS WHERE station_id = sp.station_to),
           bl.geometry
        ) * 111320 * COS(RADIANS(41.4)) <= 150
),
```

Op3. Peso compuesto

El peso final de cada arista se calcula mediante una proyección con expresión condicional sobre el **outer join** entre station_pairs y lane_affinity:

$$weighted_edges = \pi_{station_from,\, station_to,\, \omega \to distance_m}(station_pairs \bowtie lane_affinity)$$

donde la función de peso ω integra los tres factores del modelo:

$$\omega(d, \ell, \Delta h) = d \times \{1,0\ si\ \ell\ (afinidad\ ciclista),\ 1,5\ si\ \neg\ell\ (sin\ carril\ compartido)\} + max(\Delta h,\ 0) \times 0,1$$

El término aditivo penaliza el desnivel positivo (cuesta arriba) y no penaliza el negativo, lo que recoge asimétricamente el efecto del relieve sobre los flujos.

```sql
-- Operación 3: peso compuesto (distancia × factor ciclista + penalización desnivel)
weighted_edges AS (
    SELECT
        sp.station_from,
        sp.station_to,
        -- hemos tenido que aplicar una condicional
        ROUND(sp.dist_direct_m * CASE WHEN la.station_from IS NOT NULL THEN 1.0 ELSE 1.5 END
        + GREATEST(sp.altitude_diff_m, 0) * 0.1, 2) AS distance_m
    FROM      station_pairs    sp
    LEFT JOIN lane_affinity    la
           ON la.station_from = sp.station_from
          AND la.station_to   = sp.station_to
)

SELECT station_from, station_to, distance_m
FROM   weighted_edges
ORDER  BY station_from, distance_m;
```

5. Transformación del Modelo ER en Tablas del Modelo Relacional

La conversión del modelo conceptual (ER) en el modelo relacional se hace siguiendo el proceso estándar. El resultado es un conjunto de cuatro tablas interrelacionadas que recogen la totalidad de la información del sistema.

La tabla stations almacena los atributos estáticos de cada estación. Su clave primaria es station_id, de tipo entero. Las columnas latitude, longitude y altitude recogen la posición geográfica y la cota sobre el nivel del mar. La columna capacity indica el número máximo de anclajes disponibles. Las columnas district_id, district_name, neighborhood_id y neighborhood_name recogen los atributos territoriales descompuestos durante la fase de transformación.

La tabla station_status almacena las observaciones temporales del estado operativo. Su clave primaria compuesta está formada por station_id y recorded_at, que es la marca temporal de la observación. Las columnas bikes_available, docks_available, mechanical_bikes y electrical_bikes recogen los valores operativos en el instante de la observación. La columna is_operative indica si la estación estaba en funcionamiento y station_id actúa como clave foránea hacia la tabla stations.

La tabla bike_lanes almacena los segmentos de infraestructura ciclable. Su clave primaria es lane_id. Las columnas lane_type y lane_name recogen los atributos descriptivos del segmento presentes en el GeoJSON. La columna geometry almacena la geometría de tipo LINESTRING en el sistema de referencia WGS84, lo que permite el cálculo de distancias y proyecciones mediante las funciones espaciales de MySQL.

La tabla station_edges materializa el grafo de adyacencia. Sus columnas station_from y station_to forman la clave primaria compuesta y actúan ambas como claves foráneas hacia station. La columna distance_m almacena el peso de la arista en metros. Esta tabla no procede de ninguna de las fuentes originales, sino que se genera a partir del análisis geoespacial de la red de carriles bici y las posiciones de las estaciones.

STATIONS(station_id, name, latitude, longitude, altitude, capacity, district_id, district_name, neighborhood_id, neighborhood_name)

- station_id: INTEGER, clave primaria, único, no nulo
- name: VARCHAR(100), no nulo
- latitude: DECIMAL(9,6), no nulo
- longitude: DECIMAL(9,6), no nulo
- altitude: DECIMAL(6,2), nullable
- capacity: INTEGER, no nulo
- district_id: INTEGER, no nulo
- district_name: VARCHAR(100), no nulo
- neighborhood_id: INTEGER, no nulo
- neighborhood_name: VARCHAR(100), no nulo

STATION_STATUS(*station_id*, recorded_at, bikes_available, docks_available, mechanical_bikes, electrical_bikes, is_operative)

- station_id: INTEGER, clave primaria y clave foránea → STATIONS(station_id), no nulo
- recorded_at: DATETIME, clave primaria, no nulo
- bikes_available: INTEGER, no nulo
- docks_available: INTEGER, no nulo
- mechanical_bikes: INTEGER, no nulo
- electrical_bikes: INTEGER, no nulo
- is_operative: BOOLEAN, no nulo

BIKE_LANES(lane_id, lane_type, lane_name, geometry)

- lane_id: INTEGER, clave primaria, único, no nulo
- lane_type: VARCHAR(50), nullable
- lane_name: VARCHAR(100), nullable
- geometry: LINESTRING (SRID 4326), no nulo

STATION_EDGES(*station_from*, *station_to*, distance_m)

- station_from: INTEGER, clave primaria y clave foránea → STATIONS(station_id), no nulo
- station_to: INTEGER, clave primaria y clave foránea → STATIONS(station_id), no nulo
- distance_m: DECIMAL(10,2), no nulo

6. Álgebra Relacional.

Las consultas en álgebra relacional se han utilizado en la creación del grafo (Sección 4.2) y en el análisis de datos (Sección 11).

7. Predicción de ocupación mediante difusión en grafo

El grafo de estaciones queda definido formalmente por un conjunto de nodos V (las estaciones), un conjunto de aristas E derivado de STATION_EDGES y una función de peso $w: E \rightarrow \mathbb{R}^+$ que asigna a cada par de estaciones conectadas la distancia compuesta calculada en la sección anterior.

Para el modelo predictivo se adopta una formulación discreta de difusión sobre el grafo. Sea x(t) el vector de ocupación normalizada de las N estaciones en el instante t, donde cada componente se define como:

$$x_i(t) = \frac{\text{bikes_available}_i(t)}{\text{capacity}_i} \in [0, 1]$$

Los pesos de las aristas se transforman mediante un kernel gaussiano para que la influencia decrezca suavemente con la distancia y se anule de forma práctica a partir de varias desviaciones estándar:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{2\sigma^2}\right)$$

donde d_ij es el valor de distance_m en STATION_EDGES y σ es un parámetro de escala calibrable. Con estos pesos se construye la matriz de transición T normalizando por filas, de modo que cada fila suma 1 y T actúa como promedio ponderado de la vecindad. La evolución del sistema entre dos instantes consecutivos se aproxima mediante:

$$x(t+1) = \alpha\, x(t) + (1-\alpha)\, T\, x(t)$$

donde α ∈ (0,1) es el parámetro de inercia. De tal forma que valores próximos a 1 confían en el estado propio de cada estación, mientras que valores próximos a 0 dejan que la predicción quede dominada por la vecindad. El término de difusión $(1-\alpha)\, T\, x(t)$ sube la predicción de una estación cuando sus vecinas tienen mayor ocupación media y la baja en caso contrario, comportamiento que parece coherente con la dinámica esperada del sistema.

8. Normalización del Modelo Relacional

El modelo relacional propuesto ha sido diseñado siguiendo los principios de normalización con el objetivo de evitar redundancias, garantizar la integridad de los datos y mejorar la eficiencia de las consultas.

La tabla STATIONS se encuentra en Tercera Forma Normal (3FN), ya que todos sus atributos dependen funcionalmente de la clave primaria (*station_id*) y no existen dependencias transitivas. Los atributos territoriales han sido descompuestos en identificadores y nombres para evitar redundancias.

La tabla STATION_STATUS también cumple la 3FN. Su clave compuesta (*station_id, recorded_at*) determina completamente el resto de atributos, y no existen dependencias parciales ni transitivas.

La tabla BIKE_LANES se encuentra igualmente en 3FN, ya que el atributo *lane_id* identifica de manera única cada registro, y el resto de atributos dependen exclusivamente de él.

Por último, la tabla STATION_EDGES cumple la 3FN, dado que su clave compuesta (*station_from, station_to*) determina el atributo *distance_m*.

En conjunto, el modelo garantiza consistencia, elimina redundancias innecesarias y facilita la escalabilidad del sistema.

9. Implementación
9.1 Arquitectura del Pipeline ETL

El proceso de integración y carga de datos se ha implementado mediante una arquitectura basada en Python, organizada en cinco scripts modulares con responsabilidades separadas. Esta estructura asegura la correcta transformación de las fuentes originales (JSON y GeoJSON) en consultas SQL para su posterior inserción en la base de datos MySQL.

Dentro de esta estructura, el script `config.py` se encarga de centralizar las URLs de las fuentes de datos, el token de autenticación del portal Open Data Barcelona y las credenciales de conexión a la base de datos MySQL. Por su parte, `fetcher.py` implementa la descarga de datos mediante peticiones HTTP GET, añadiendo a cada una de ellas la cabecera de autorización necesaria para acceder al portal de forma segura.

La transformación de la información recae en los scripts `etl_station_information.py`, `etl_station_status.py` y `etl_carril_bici.py`, los cuales leen cada fuente de datos respectiva con el objetivo de aplanar las estructuras anidadas y generar las sentencias SQL de inserción. Finalmente, `run_etl.py` funciona como el orquestador de todo el proceso, encargándose de encadenar los distintos módulos de extracción y transformación en el orden correcto y de ejecutar la carga final de los datos en MySQL.

9.2 Orden de Carga y Dependencias

Para garantizar la integridad referencial de la base de datos, el orquestador *run_etl.py* respeta estrictamente un orden de carga predefinido basado en las dependencias de clave foránea. El proceso comienza con la carga de *STATIONS*, que se introduce en primer lugar al ser la entidad principal y no tener dependencias externas. Una vez completado este paso, se inserta la información de *STATION_STATUS*, ya que esta tabla contiene una clave foránea que referencia directamente a la tabla principal de estaciones.

Por otro lado, la tabla *BIKE_LANES* se carga de forma totalmente independiente, puesto que carece de dependencias directas con el resto de componentes del modelo de datos. Por último, la entidad *STATION_EDGES* se genera al final de todo el flujo de trabajo, debido a que sus atributos *station_from* y *station_to* actúan como claves foráneas que apuntan necesariamente hacia *STATIONS*.

9.3 Decisiones de Transformación Clave

Durante el desarrollo del pipeline, se han aplicado dos transformaciones esenciales para optimizar el rendimiento del análisis del sistema. La primera de ellas consiste en el aplanamiento de datos JSON dentro del dataset de estado de las estaciones. En la fuente original, el desglose por tipo de bicicleta viene agrupado bajo la clave anidada *num_bikes_available_types*, que contiene los campos *mechanical* y *ebike*. En lugar de almacenar un objeto JSON que requeriría la función *JSON_EXTRACT()* en tiempo de ejecución para cada consulta, estos valores se extraen durante la fase de transformación y se guardan como columnas escalares independientes denominadas *mechanical_bikes* y *electrical_bikes*, lo que permite un acceso directo sin coste computacional adicional de parseo.

La segunda transformación clave es la definición explícita del SRID 4326 (WGS84) tanto para las coordenadas de las estaciones como para las geometrías de los carriles bici al momento de su inserción. Esta especificación es indispensable para asegurar que las funciones espaciales de MySQL calculen las distancias en metros reales y no en grados. De este modo, se garantiza que operaciones espaciales críticas como *ST_Distance* o *ST_Intersection* se ejecuten de manera eficiente y produzcan resultados métricos correctos para el análisis.

9.4 Ejecución del Sistema

El despliegue del sistema completo desde cero se realiza mediante la siguiente secuencia de comandos en la terminal, los cuales configuran el entorno, crean el esquema y pueblan la base de datos:

1. Instalación de las dependencias de Python:
   `pip3 install requests mysql-connector-python`
2. Creación del esquema de la base de datos (solo la primera vez):
   `mysql -u root -p < schema.sql`
3. Carga de los datos ejecutando el pipeline ETL:
   `python3 run_etl.py --out-dir ./sql_output`
4. Construcción del grafo de estaciones sobre la red ciclista:
   `mysql -u root -p bicing < station_edges_graph.sql`

10. Análisis de los Datos
10.1 Operaciones Relacionales del Modelo Predictivo

Queremos transformar la esencia del proceso de difusión en un grafo (sección 7) mediante consultas de SQL.
La evaluación del modelo se articula en cuatro operaciones encadenadas:

Op1. Dos snapshots por estación

El modelo necesita no solo el estado actual de cada estación sino también el estado anterior, para capturar la tendencia temporal. Se utiliza una función de ventana ROW_NUMBER() particionada por estación y ordenada por marca temporal descendente, que asigna el rango 1 al snapshot más reciente y el rango 2 al inmediatamente anterior. Sobre ese ranking se construyen dos relaciones independientes:

$$\text{PREV} = \pi_{\text{station_id, bikes_available/capacity} \to x_i^{\text{prev}}} (\sigma_{\text{rn}=2}(\text{RANKED}) \bowtie S) \quad S)$$

donde RANKED es el resultado de aplicar ROW_NUMBER() sobre STATION_STATUS. La ocupación normalizada se define en ambos casos como el cociente entre bicicletas disponibles y capacidad total de la estación, restringido a estaciones con capacidad positiva para evitar dividir por cero.

```sql
-- Operación 1: dos snapshots por estación (actual y anterior)
ranked_status AS (
    SELECT
        ss.station_id,
        ss.recorded_at,
        ss.bikes_available,
        ROW_NUMBER() OVER (
            PARTITION BY ss.station_id
            ORDER BY ss.recorded_at DESC
        ) AS rn
    FROM STATION_STATUS ss
),
current_occupancy AS (
    SELECT
        rs.station_id,
        rs.bikes_available / s.capacity AS occupancy
    FROM ranked_status rs
    JOIN STATIONS s USING (station_id)
    WHERE rs.rn = 1
      AND s.capacity > 0
),
previous_occupancy AS (
    SELECT
        rs.station_id,
        rs.bikes_available / s.capacity AS occupancy_prev
    FROM ranked_status rs
    JOIN STATIONS s USING (station_id)
    WHERE rs.rn = 2
      AND s.capacity > 0
),
```

Op2. Pesos gaussianos sobre aristas simétricas

Antes de aplicar el kernel gaussiano [ver memoria parcial, o el apartado (7) sobre difusión en un grafo], las aristas de STATION_EDGES se simetrizan mediante una unión con los pares invertidos, ya que la tabla almacena cada arista una sola vez en sentido canónico (station_from < station_to). Esta simetría es lo que garantiza que cada estación pueda ser “influenciada” por todas sus vecinas.

$$\mathrm{SYM} = \mathrm{STATION_EDGES} \cup \pi_{\mathrm{station_to}\to\mathrm{station_from},\ \mathrm{station_from}\to\mathrm{station_to},\ \mathrm{distance_m}}(\mathrm{STATION_EDGES})$$

Sobre el conjunto simétrico se calcula el peso gaussiano de cada arista:

$$W = \pi_{\mathrm{station_from},\ \mathrm{station_to},\ \exp(-d_{ij}^2/2\sigma^2)\to w_{ij}}(\mathrm{SYM})$$

Con σ = 300m, una estación a 300 m recibe un peso de $e^{-0.5} \approx 0,61$ y una estación a 600 m recibe $e^{-2} \approx 0,14$. De este modo, la influencia decae suavemente y se anula de forma práctica más allá de dos o tres desviaciones estándar.

```
-- Operación 2: pesos gaussianos sobre aristas en ambos sentidos
sym_edges AS (
    SELECT station_from, station_to, distance_m
    FROM STATION_EDGES
    UNION ALL
    SELECT station_to AS station_from, station_from AS station_to, distance_m
    FROM STATION_EDGES
),
gaussian_weights AS (
    SELECT
        station_from,
        station_to,
        EXP(-(distance_m * distance_m) / (2 * @sigma * @sigma))  AS w
    FROM sym_edges
),
```

Op3. Normalización por filas (matriz de transición T)

Para que los pesos actúen como una media ponderada de la vecindad y no como una suma acumulada, se normalizan por filas dividiendo cada peso por la suma total de pesos salientes de cada estación:

$$W_{\text{norm}} = \pi_{\text{station_from, station_to, } w_{ij}/\sum_k w_{ik} \to t_{ij}} \left(W \bowtie_{\text{station_from}} \gamma_{\text{station_from; SUM}(w)\to\text{total}}(W)\right)$$

El resultado es la matriz de transición T, donde cada fila suma exactamente 1. Cada entrada t_ij expresa qué fracción de la influencia total recibida por la estación i proviene de la estación j.

```
-- Operación 3: normalización por filas -> matriz de transición T
row_totals AS (
    SELECT station_from, SUM(w) AS total_w
    FROM gaussian_weights
    GROUP BY station_from
),
transition AS (
    SELECT
        gw.station_from,
        gw.station_to,
        gw.w / rt.total_w  AS t_ij
    FROM gaussian_weights gw
    JOIN row_totals rt ON rt.station_from = gw.station_from
),
```

Op4. Predicción

La predicción combina tres señales con pesos independientes α, β y (1-α-β), que en la configuración actual toman los valores 0,20, 0,45 y 0,35 respectivamente.

La primera señal es la influencia de la vecindad espacial. Tras simetrizar las aristas y normalizar los pesos, se calcula para cada estación i la suma ponderada de las ocupaciones actuales de sus vecinas:

$$\text{INFLUENCE} = \gamma_{\text{station_from};\text{SUM}(t_{ij}\cdot x_j)\to\hat{x}_{nb}} \left(W_{\text{norm}} \bowtie_{\text{station_to}=\text{station_id}} \text{CURRENT}\right)$$

El operador γ agrupa por estación de origen y acumula los productos t_{ij} · x_{j} sobre todos los vecinos j de cada estación i, produciendo un único valor

$\hat{x_{nb,i}}$

por estación: la ocupación media ponderada de su vecindad.
Dado que la matriz de transición está normalizada por filas, este valor es siempre una media convexa de las ocupaciones vecinas, es decir, toma valores en [0,1] y las estaciones más próximas contribuyen proporcionalmente más que las lejanas.

```
-- Operación 4a: influencia de la vecindad (T · x(t))
neighbor_influence AS (
    SELECT
        t.station_from  AS station_id,
        SUM(t.t_ij * co.occupancy) AS nb_occupancy
    FROM transition t
    JOIN current_occupancy co ON co.station_id = t.station_to
    GROUP BY t.station_from
),
```

La segunda señal es la tendencia temporal propia de cada estación, definida como la variación observada entre los dos últimos snapshots disponibles:

$$\Delta x_i = x_i(t) - x_i(t-1)$$

Esta señal recoge si la estación se está llenando o vaciando en el intervalo más reciente, independientemente del estado de sus vecinas.

Esta diferencia es positiva si la estación se estaba llenando en el intervalo más reciente y negativa si se estaba vaciando. Proyectarla hacia adelante como xi(t)+Δxi equivale a asumir que la tendencia inmediata se mantiene en el siguiente intervalo, lo que es una aproximación razonable para horizontes cortos. Se utiliza un LEFT JOIN con PREV para que las estaciones que solo tienen un snapshot no queden excluidas del resultado. En ese caso, Δxi=0 y la señal temporal no aporta ni penaliza.

```
-- Operación 4b: tendencia temporal por estación (x(t) - x(t-1))
temporal_signal AS (
    SELECT
        co.station_id,
        co.occupancy,
        po.occupancy_prev,
        (co.occupancy - COALESCE(po.occupancy_prev, co.occupancy)) AS temporal_delta
    FROM current_occupancy co
    LEFT JOIN previous_occupancy po USING (station_id)
),
```

La predicción final integra las tres señales mediante una combinación lineal convexa, ya que los coeficientes suman 1 por construcción:

$$\text{PRED} = \pi_{\text{station_id},\ \alpha \cdot x_i + \beta \cdot \hat{x}_{\text{nb}} + (1-\alpha-\beta)\cdot(x_i + \Delta x_i) \to x_i(t+1)} \left(\text{TS} \bowtie \text{INFLUENCE}\right)$$

Con los parámetros actuales (α = 0,20, β = 0,45, 1-α-β = 0,35) el término dominante es la vecindad espacial, lo que refleja la hipótesis central del proyecto de que las estaciones próximas intercambian demanda y tienden a equilibrarse. El estado actual propio actúa como “ancla” que evita que la predicción se aleje demasiado de la observación presente y la tendencia temporal introduce una corrección direccional que distingue entre una estación estable en el 50% de ocupación y una que está llegando al 50% desde arriba o desde abajo.

El resultado se acota al intervalo [0,1] mediante GREATEST(0, LEAST(1, ...)) para garantizar que la ocupación predicha sea siempre una proporción válida, evitando artefactos numéricos en estaciones con ocupación extrema o tendencias muy pronunciadas.

```sql
-- Operación 4c: predicción espacial-temporal
-- x(t+1) = α·x(t) + β·T·x(t) + (1-α-β)·(x(t) + (x(t)-x(t-1)))
prediction AS (
    SELECT
        ts.station_id,
        ROUND(
            LEAST(
                1.0,
                GREATEST(
                    0.0,
                    @alpha * ts.occupancy
                    + @beta * COALESCE(ni.nb_occupancy, ts.occupancy)
                    + (1 - @alpha - @beta) * (ts.occupancy + ts.temporal_delta)
                )
            ),
            4
        ) AS predicted_occupancy
    FROM temporal_signal ts
    LEFT JOIN neighbor_influence ni ON ni.station_id = ts.station_id
)
```

Interpretación de los parámetros y el resultado

```sql
-- variables globales (se pueden ajustar)
SET @sigma  = 300.0;   -- escala gaussiana en metros
SET @alpha  = 0.20;    -- peso de ocupación actual
SET @beta   = 0.45;    -- peso de vecindad espacial
SET @t_star = (SELECT MAX(recorded_at) FROM STATION_STATUS);
```

```sql
-- Resultado final con contexto de estación
SELECT
    p.station_id,
    s.name,
    s.district_name,
    s.altitude,
    ROUND(co.occupancy, 4)                        AS current_occupancy,
    p.predicted_occupancy,
    ROUND(p.predicted_occupancy - co.occupancy, 4) AS delta,
    CASE
        WHEN p.predicted_occupancy > co.occupancy + 0.01 THEN 'SUBE'
        WHEN p.predicted_occupancy < co.occupancy - 0.01 THEN 'BAJA'
        ELSE 'ESTABLE'
    END                                            AS tendencia
FROM prediction p
JOIN current_occupancy co USING (station_id)
JOIN STATIONS s           USING (station_id)
ORDER BY p.station_id; -- para que salgan ordenadas por id
```

El parámetro α = 0,20 pondera la ocupación actual de la propia estación como “ancla” del estado presente. El parámetro β = 0,45 es el peso dominante y asigna la mayor influencia a la vecindad espacial, reflejando la hipótesis central del proyecto de que las estaciones próximas intercambian demanda. El término (1-α-β) = 0,35 pondera la proyección de la tendencia temporal propia, de modo que si una estación estaba ganando bicicletas en el último intervalo, la predicción continúa esa tendencia moderadamente.

La columna **delta** del resultado es positiva cuando la ocupación predicha supera la actual (lo que indica que la estación tenderá a recibir más bicicletas de las que emite) y negativa en el caso contrario. La columna **tendencia** clasifica cualitativamente cada estación con un umbral de 0,01 sobre la ocupación normalizada, equivalente a aproximadamente una bicicleta en una estación de capacidad media.

Los tres parámetros (α, β y σ) pueden calibrarse empíricamente sobre los datos históricos de STATION_STATUS minimizando el error entre la predicción y el estado observado en el snapshot siguiente.

Para “ejecutar” una predicción se escribe en la consola:

```
> mysql -u root -p bicing < prediction.sql
```

(Para MySQL Workbench, es necesario primero seleccionar un scheme)
Para que esta sea más realista, es necesario ejecutar varias veces:

```
> python3 run_etl.py --out-dir ./sql_output
> mysql -u root -p bicing < station_edges_graph.sql
```

para añadir nuevos snapshots de las estaciones.

Por ejemplo, con estos snapshots:

station_id	recorded_at	bikes_available	docks_available	mechanical_bik...	electrical_bik...	is_operative
1	2026-04-27 13:38:06	6	34	5	1	1
1	2026-04-27 16:35:44	7	34	4	3	1
1	2026-04-27 16:44:47	2	39	1	1	1
2	2026-04-27 13:37:57	10	16	2	8	1
2	2026-04-27 16:35:38	2	24	2	0	1
2	2026-04-27 16:44:39	2	24	2	0	1
3	2026-04-27 13:38:51	4	21	0	4	1
3	2026-04-27 16:36:46	8	17	7	1	1
3	2026-04-27 16:45:48	8	16	7	1	1

obtenemos:

station_id	name	district_na...	altitude	current_occupancy	predicted_occupancy	delta	tendencia
1	GRAN VIA CORTS CATALANES, 760		16.00	0.0435	0.0054	-0.0381	BAJA
2	C/ ROGER DE FLOR, 126		17.00	0.0690	0.069	0	ESTABLE
3	C/ NÀPOLS, 82		11.00	0.2963	0.2963	0	ESTABLE
4	C/ RIBES, 13		8.00	0.5238	0.5738	0.05	SUBE
5	PG. LLUIS COMPANYS, 11 (ARC TRIOMF)		7.00	0.3056	0.3153	0.0097	ESTABLE
6	PG. LLUIS COMPANYS, 18 (ARC TRIOMF)		10.00	0.2051	0.2051	-0	ESTABLE
7	PG. PUJADES, 1 (JUTJATS)		6.00	0.5185	0.5056	-0.0129	BAJA
8	PG. PUJADES, 2		6.00	0.0741	0.0611	-0.013	BAJA
9	AV. MARQUÉS DE L'ARGENTERA,13		5.00	0.7778	0.7778	0	ESTABLE
10	C/ 60, NÚMERO 25		4.70	0.2093	0.2093	-0	ESTABLE
11	PG. MARITIM, 11 (DAVANT PL. BRUGADA)		7.34	0.2286	0.2486	0.02	SUBE

11. Conclusiones

El proyecto ha demostrado que los datos heterogéneos del sistema Bicing pueden integrarse en un esquema relacional normalizado y servir de base para un modelo predictivo que trata la red como un sistema espacialmente interdependiente.

La base de datos, construida sobre cuatro tablas relacionales, recoge la totalidad de la información relevante: atributos estáticos de estación, observaciones temporales de estado, infraestructura ciclista en formato geoespacial y el grafo de adyacencia derivado. El pipeline ETL transforma los JSON y GeoJSON originales en filas y columnas manteniendo toda la lógica dentro del sistema relacional, lo que garantiza la reproducibilidad completa del proceso sin dependencias externas.

El aspecto que mayor evolución ha experimentado entre la entrega parcial y la versión final es la construcción del grafo. La propuesta inicial contemplaba proyectar cada estación sobre la red ciclista y calcular caminos mínimos mediante Dijkstra, opción más precisa geométricamente pero muy difícilmente expresable en álgebra relacional. La versión final la sustituye por tres operaciones SQL encadenadas: generación de pares candidatos por vecindad espacial, clasificación por afinidad ciclista mediante semijoin con BIKE_LANES, y cálculo de un peso compuesto que integra distancia y penalización por ausencia de carril compartido.

El componente predictivo ha seguido una evolución paralela. La formulación en tiempo continuo basada en el laplaciano del grafo (propuesto en la entrega parcial) no es directamente expresable en SQL. La versión final adopta una difusión discreta que combina la influencia ponderada de la vecindad espacial, el estado actual de la propia estación y la tendencia observada entre los dos últimos snapshots, con tres coeficientes calibrables empíricamente sobre los datos históricos de STATION_STATUS.

La conclusión de este proyecto es que la hipótesis de partida se sostiene: modelar cada estación como una serie temporal aislada es menos preciso que considerar su posición dentro de la red y su dinámica reciente. Los tres factores son operacionalizables dentro de un sistema

relacional estándar, sin infraestructura adicional, y el modelo resultante es directamente escalable al sistema real en la medida en que se acumulen más snapshots históricos en STATION_STATUS.

12. Contribución de los miembros del grupo y uso de la IA

Hemos utilizado IA para generar el código de los scripts para convertir de JSON a SQL, puesto que simplemente era un paso intermedio para poblar la base de datos y estaba lejos de los objetivos de la asignatura. También la hemos utilizado para recopilar información y encontrar la mejor forma de crear la difusión sobre el grafo, porqué técnicas más avanzadas (como se propusieron en la entrega parcial) resultaban muy poco prácticas a la hora de trasladarlo a consultas en álgebra relacional, y por ende a SQL. Por lo tanto, decidimos simplificar algunos aspectos (como hemos explicado en la sección 4.2).